Model comparison, intro to MCMC

Frank Edwards

How to assess a model

If our goal is causal inference

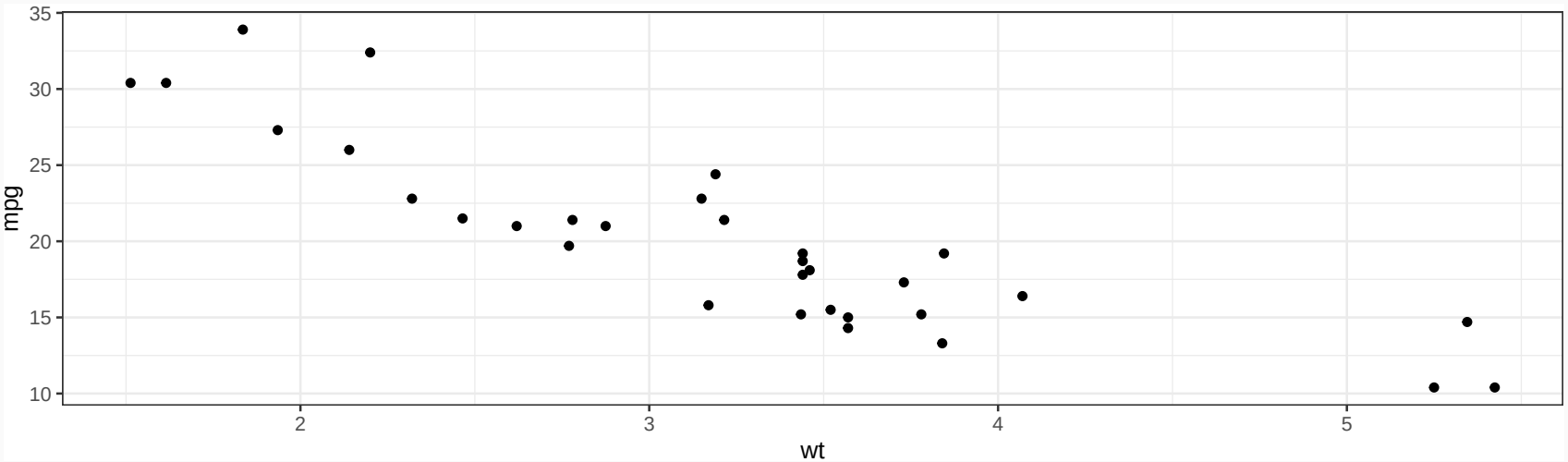
1. Construct causal theories (DAGs)
2. Determine testable implications of model
3. Evaluate, critique, refine

If our goal is prediction

1. Establish what we'd like to predict
2. Estimate competing models
3. Compare with metric of accuracy / fit

Data for today

```
data(mtcars)
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point()
```



The most common measure of model fit: R^2

$$R^2 = \frac{var(\text{outcome}) - var(\text{residuals})}{var(\text{outcome})}$$

$$R^2 = 1 - \frac{var(\text{residuals})}{var(\text{outcome})}$$

R^2 in action

```
m1<-quap(alist(
  mpg ~ dnorm(mu, sigma),
  mu<-a + bW * wt,
  a ~ dnorm(0, 10),
  bW ~ dnorm(0, 5),
  sigma ~ dexp(1)
  ), data = mtcars)
```

```
[1] 0.7494691
```

R^2 tells us how much of the variance in the outcome variable is predictable based on the included predictor variable(s). Here: we can predict 75 percent of the observed variance in mpg after knowing wt

Let’s work with a sample of the data

```
car_sample<-mtcars %>%
  sample_n(10)

d<-car_sample
```

For the linear model with weight as a predictor, $R^2 = $

```
[1] 0.794339
```

Can we improve the fit?

Yes! Let's use this linear function:

$$\mu = \alpha + \beta_1 W + \beta_2 W^2$$

$R^2 = $

```
[1] 0.8438117
```

Can we improve the fit?

Yes!

$$\mu = \alpha + \beta_1 W + \beta_2 W^2 + \beta_3 W^3$$

$R^2 = $

```
[1] 0.8584337
```

Can we improve the fit?

Yes!

$$\mu = \alpha + \beta_1 W + \beta_2 W^2 + \beta_3 W^3 + \beta_4 W^4$$

$R^2 = $

```
[1] 0.9018586
```

Can we improve the fit?

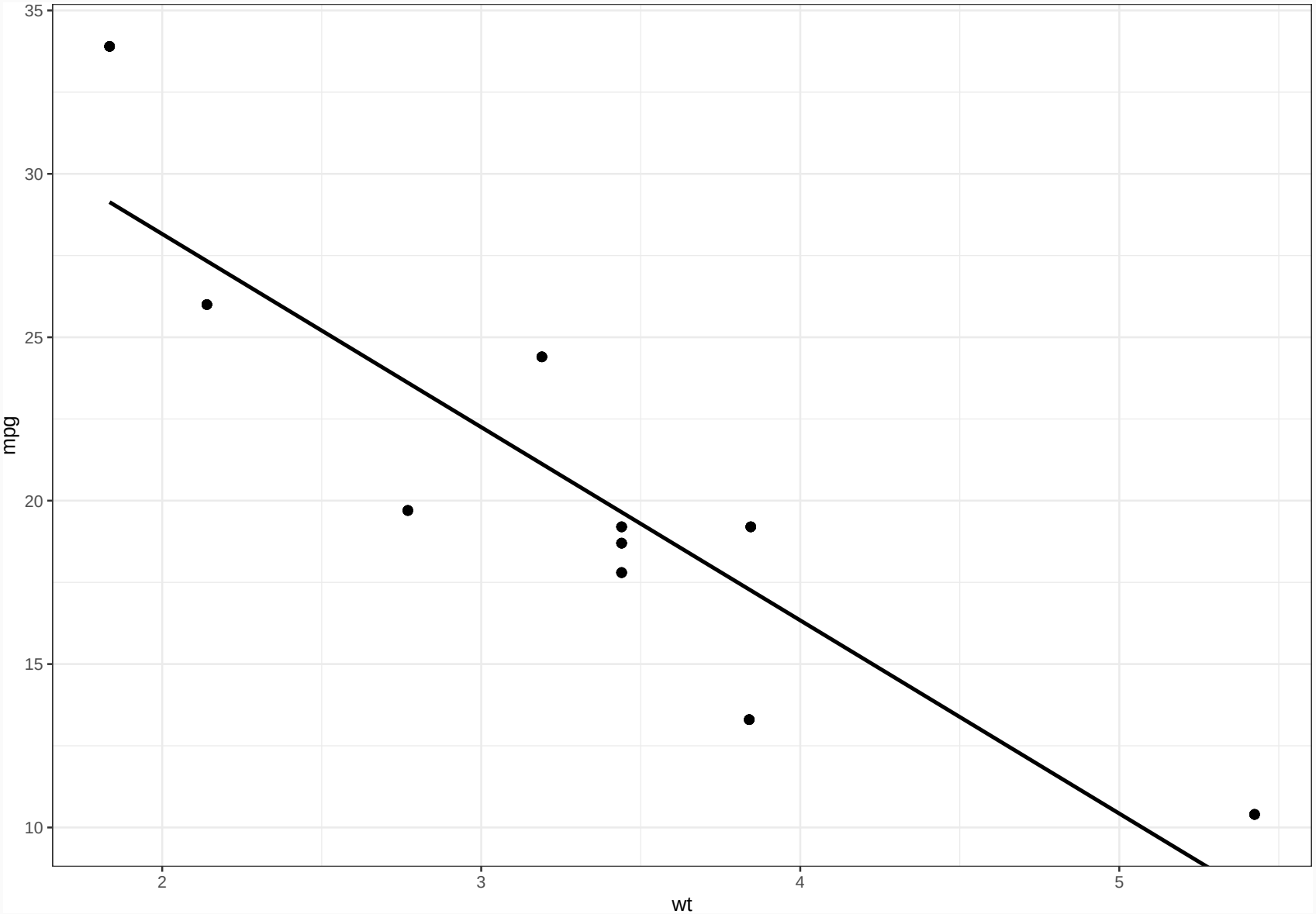
Yes!

$$\mu = \alpha + \beta_1 W + \beta_2 W^2 + \beta_3 W^3 + \beta_4 W^4 + \beta_5 W^5 + \beta_6 W^6$$

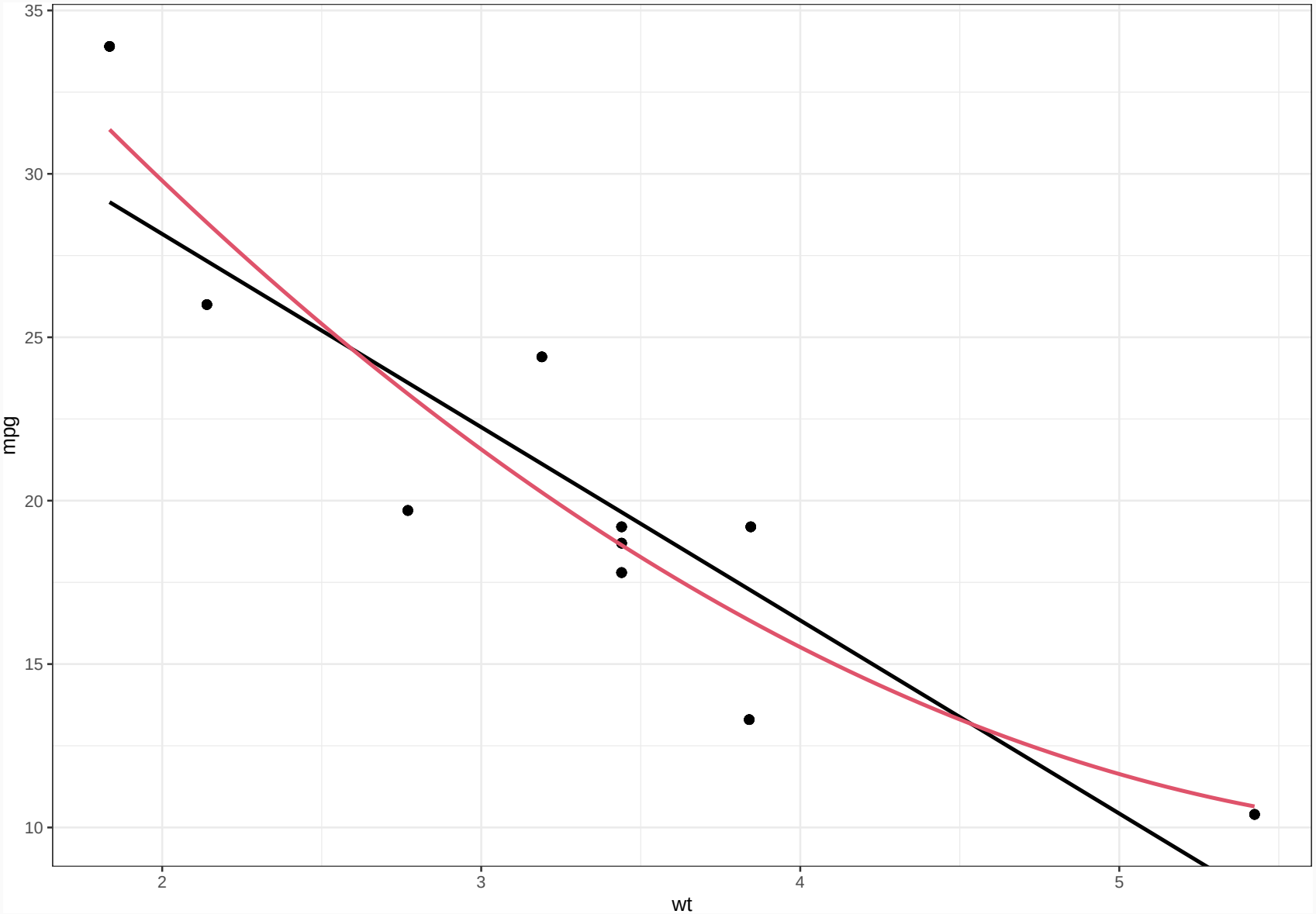
$R^2 = $

```
[1] 0.9567988
```

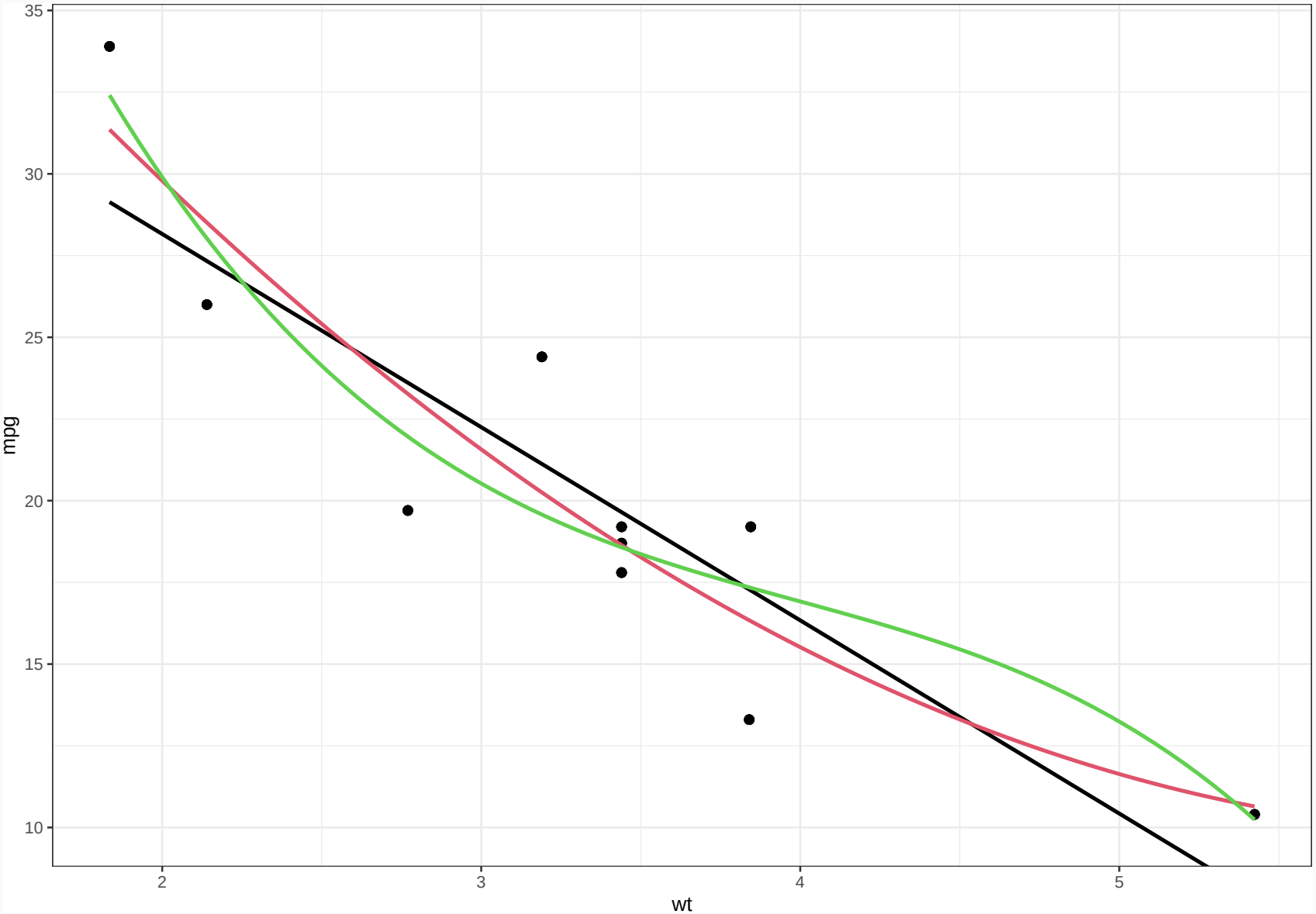
Compare these fits



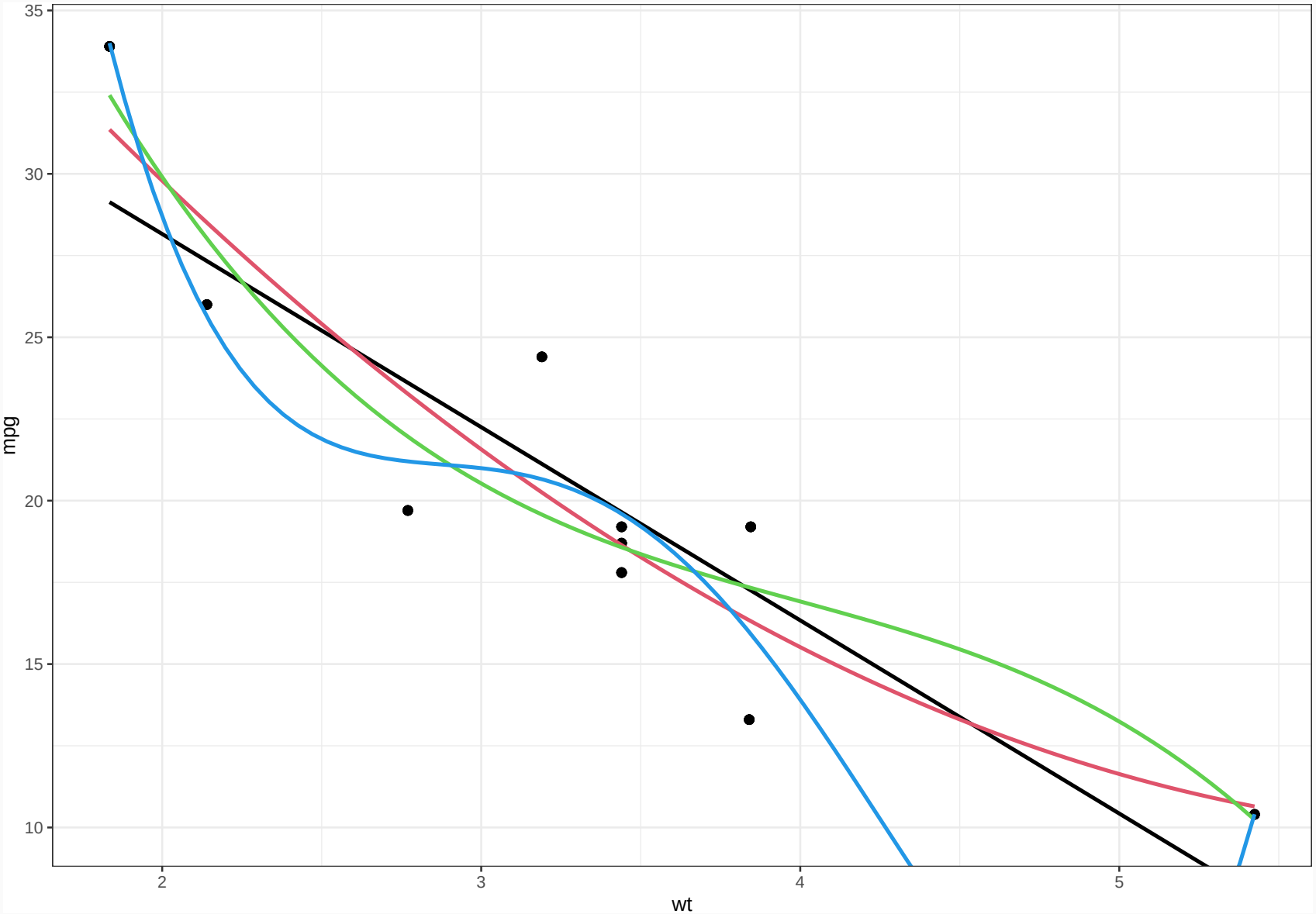
Compare these fits



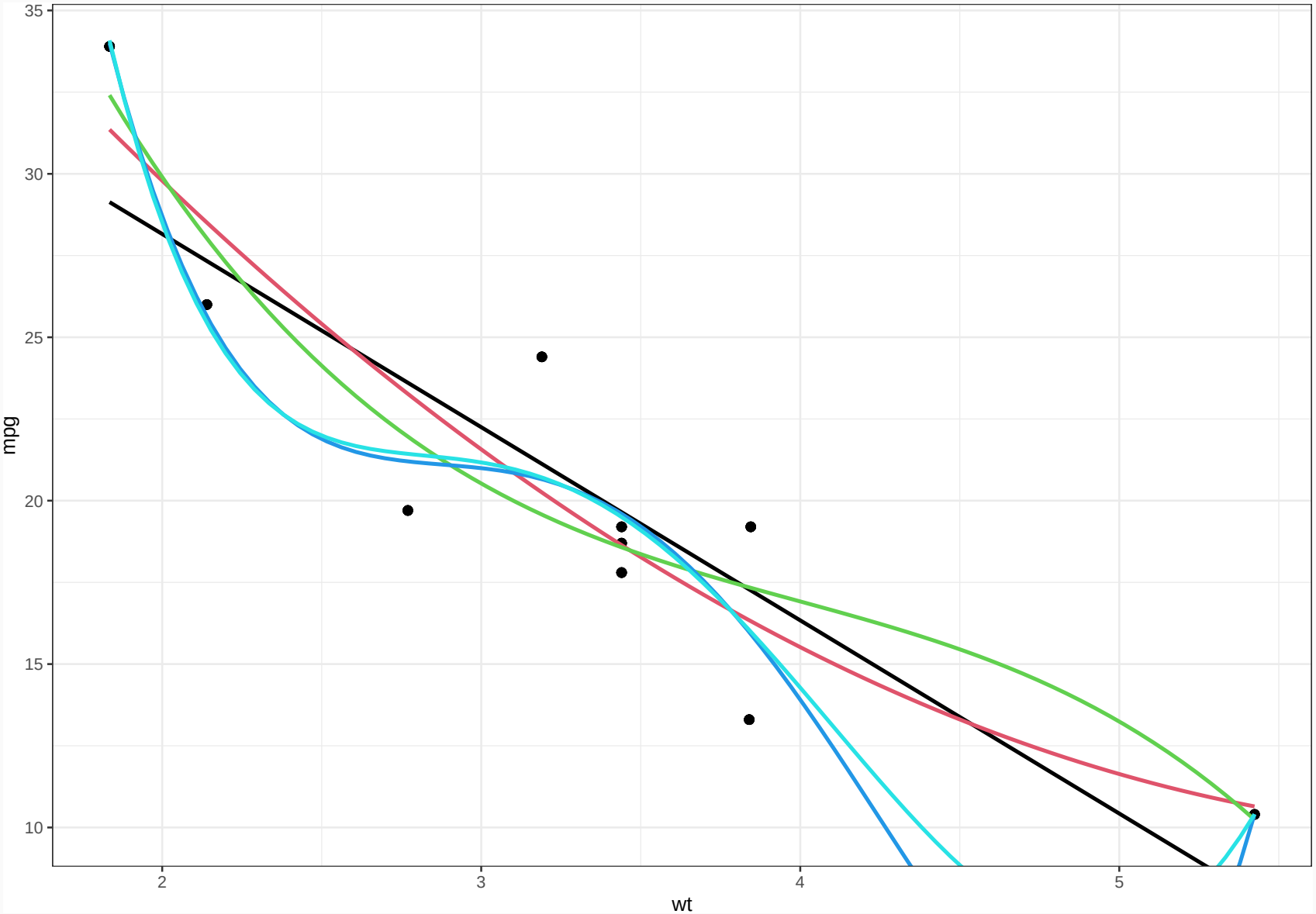
Compare these fits



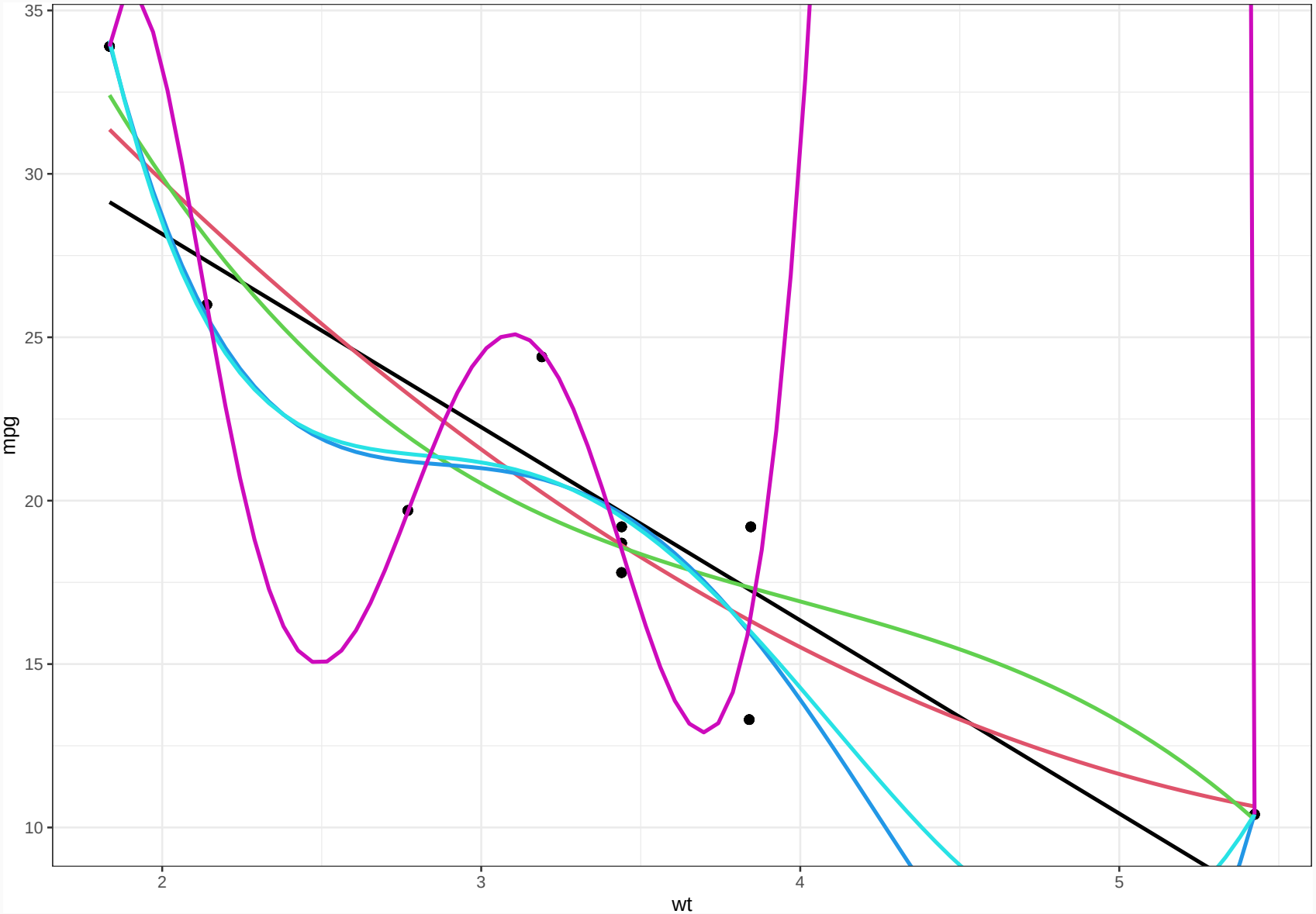
Compare these fits



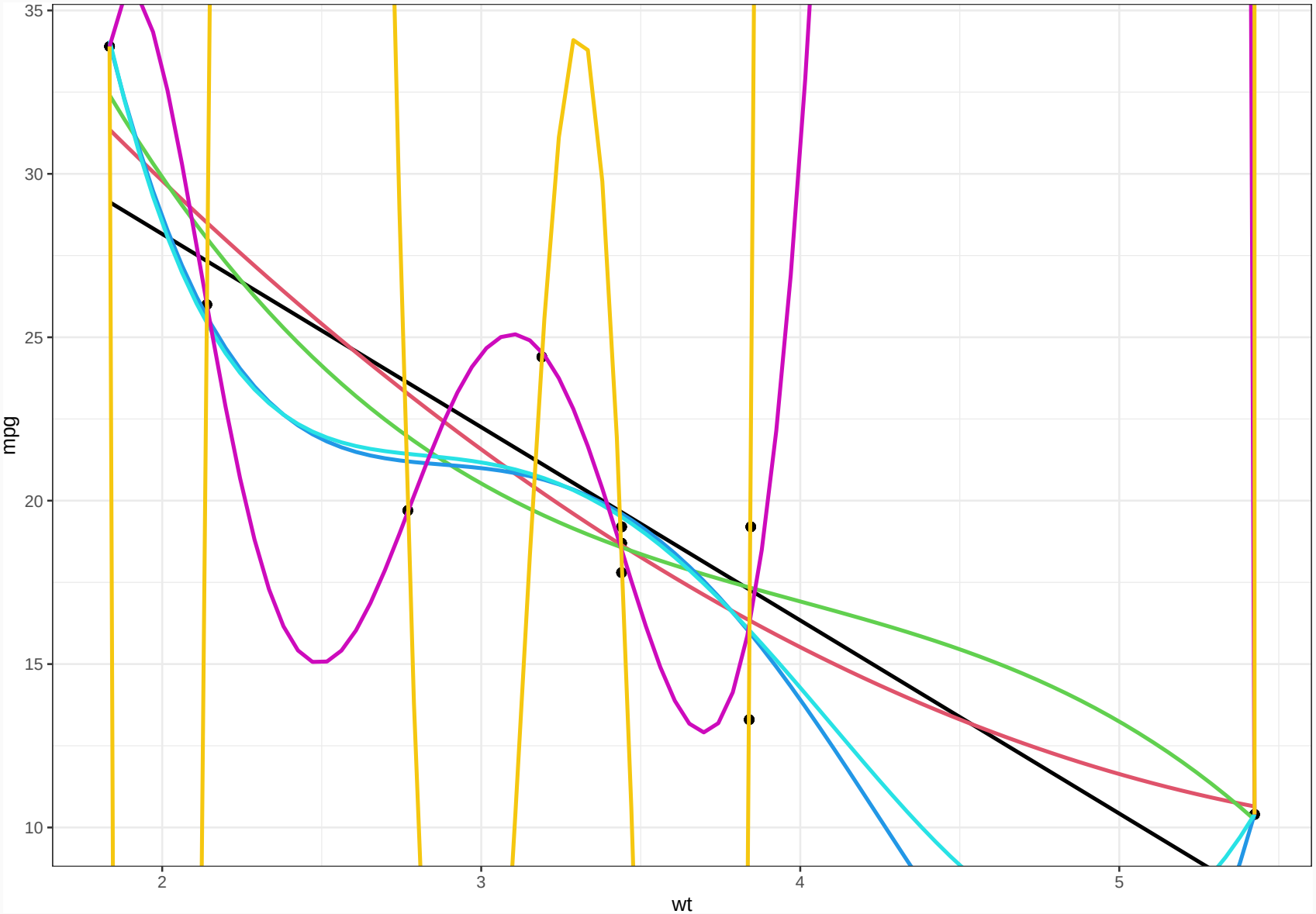
Compare these fits



Compare these fits



Compare these fits

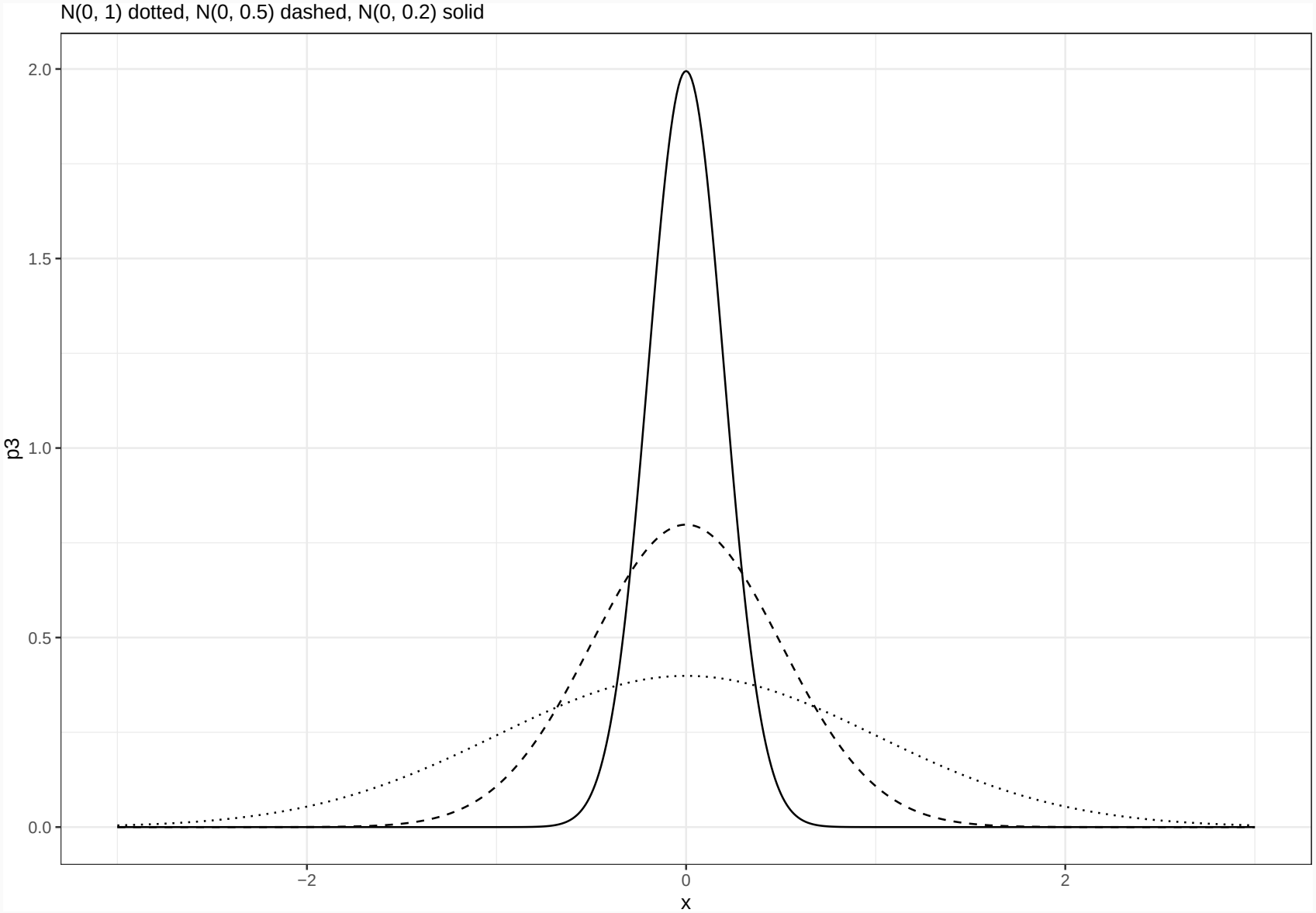


Overfitting and underfitting

- Adding complexity will improve goodness-of-fit measures like R^2
- But goodness-of-fit doesn't mean that we've modeled the process well, just that we've fit our *sample* well
- Adding too few parameters though can mean we aren't learning enough from the data
- *Goal:* balance model complexity with predictive accuracy

How to address overfitting

Regularizing priors



Use these priors to fit our polynomial model

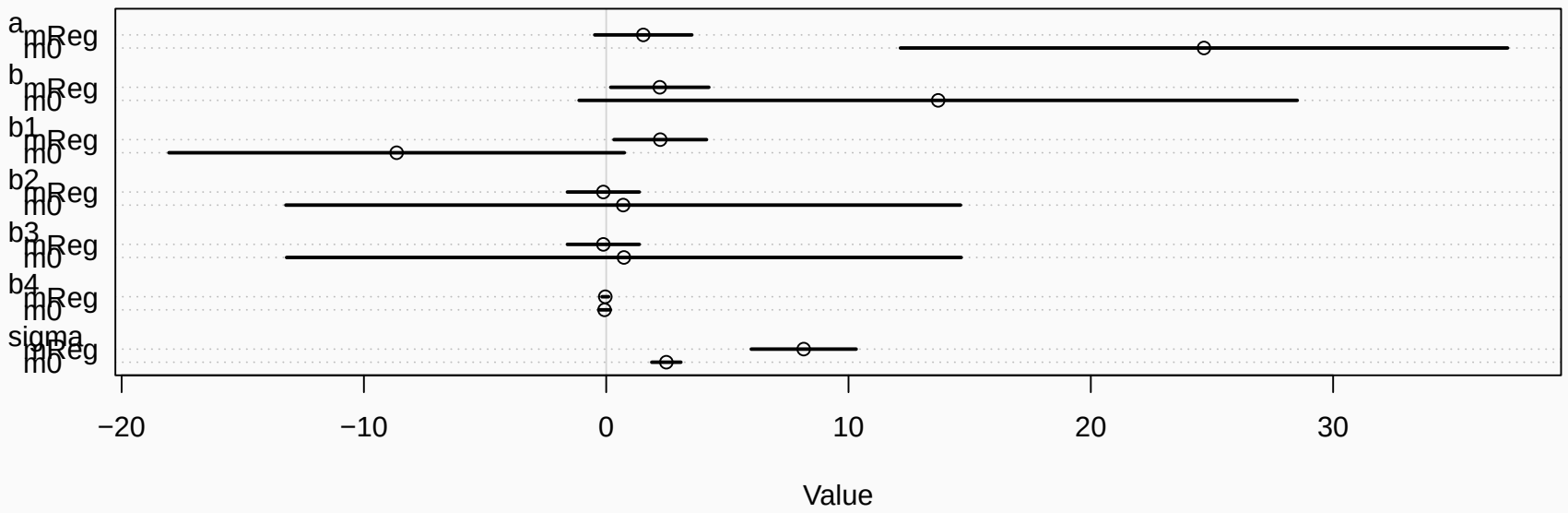
```
m0<-quap(alist(
  mpg ~ dnorm(mu, sigma),
  mu<-a + b * wt + b1 * wt^2 + b2 * wt^3 + b3 * wt^3 + b4 * wt^4,
  a ~ dnorm(0, 10),
  b ~ dnorm(0, 10),
  b1 ~ dnorm(0, 10),
  b2 ~ dnorm(0, 10),
  b3 ~ dnorm(0, 10),
  b4 ~ dnorm(0, 10),
  sigma ~ dexp(1)
), data = mtcars)

mReg<-quap(alist(
  mpg ~ dnorm(mu, sigma),
  mu<-a + b * wt + b1 * wt^2 + b2 * wt^3 + b3 * wt^3 + b4 * wt^4,
  a ~ dnorm(0, 1),
  b ~ dnorm(0, 1),
  b1 ~ dnorm(0, 1),
  b2 ~ dnorm(0, 1),
  b3 ~ dnorm(0, 1),
  b4 ~ dnorm(0, 1),
  sigma ~ dexp(1)
), data = mtcars)
```

Narrower priors constrain overfitting

These priors force the model to learn less from the data. Flat priors allow the posterior to overfit the data.

```
plot(coeftab(m0, mReg))
```



Methods for comparing models

How well can we predict cases that we didn't use to fit the model?

Out of sample prediction is a general set of methods that evaluate how well a model performs at predicting new cases.

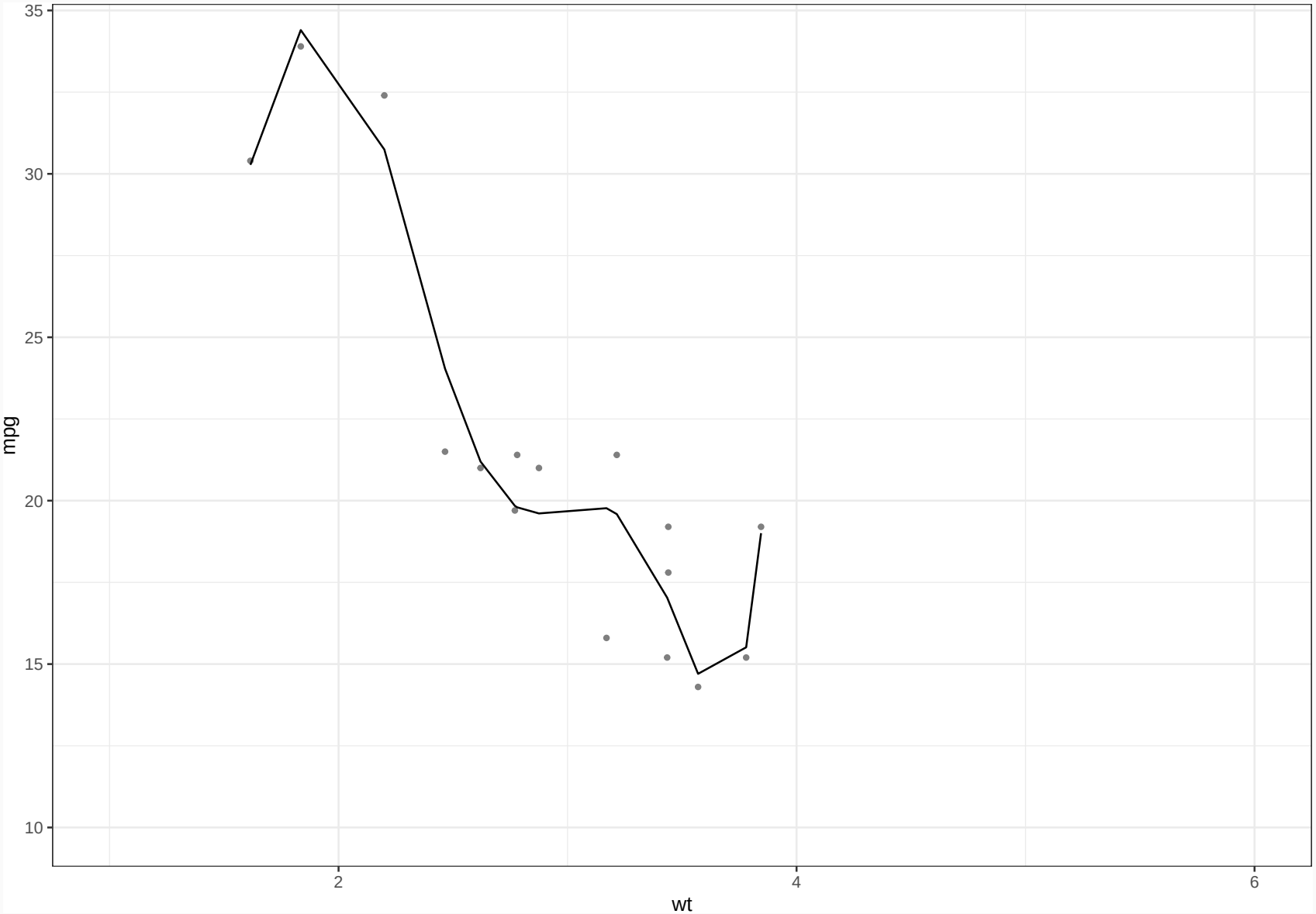
In general, we'd like to know how well our model predicts *new* observations, because it is easy to overfit on observed data.

Training and test methods

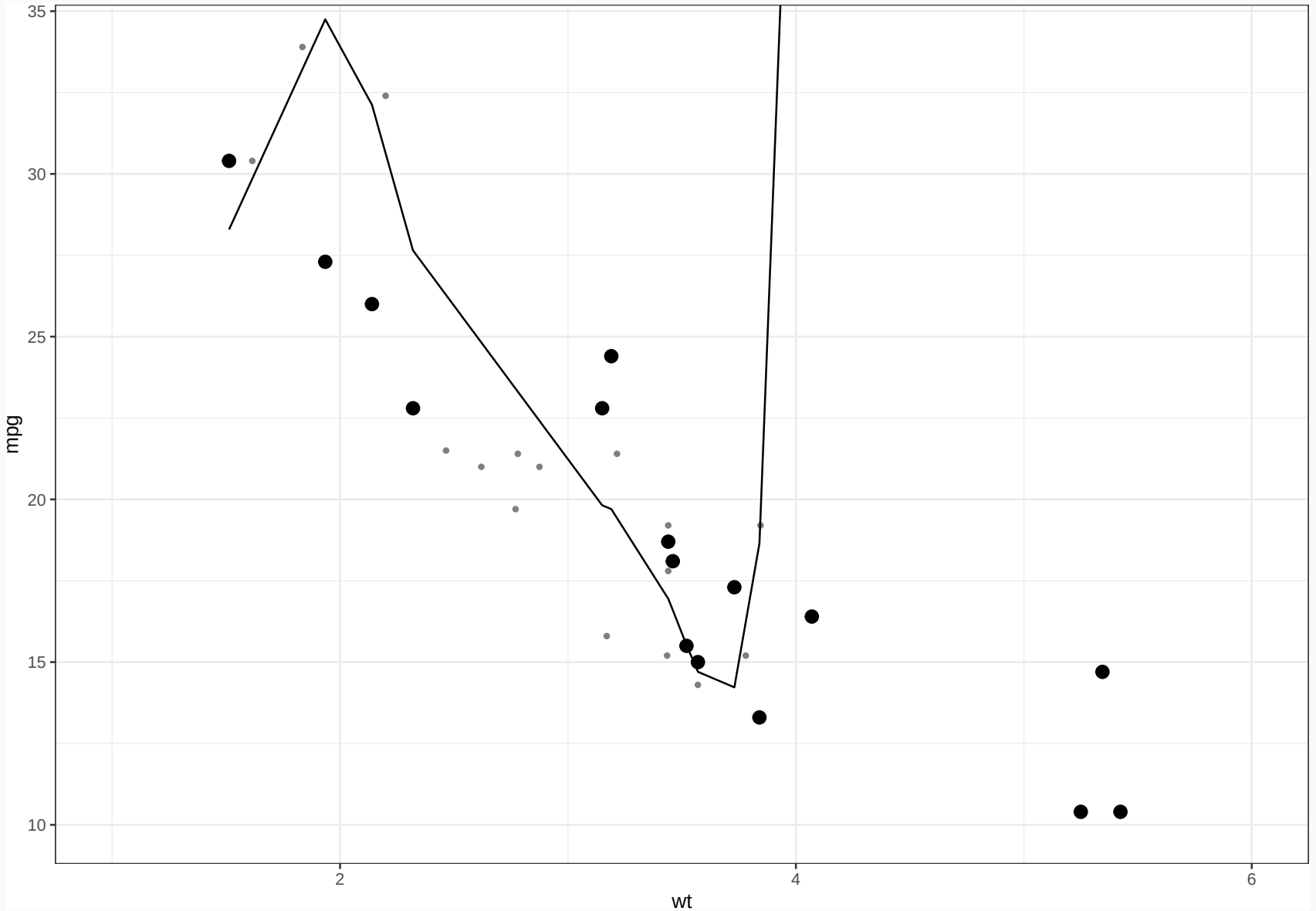
Let's partition the data into two equal sets: a *training* sample that we'll use to fit the model, and a *test* sample that we'll use to evaluate predictive accuracy

We'll extend this later to a general approach: *leave-one-out cross-validation*

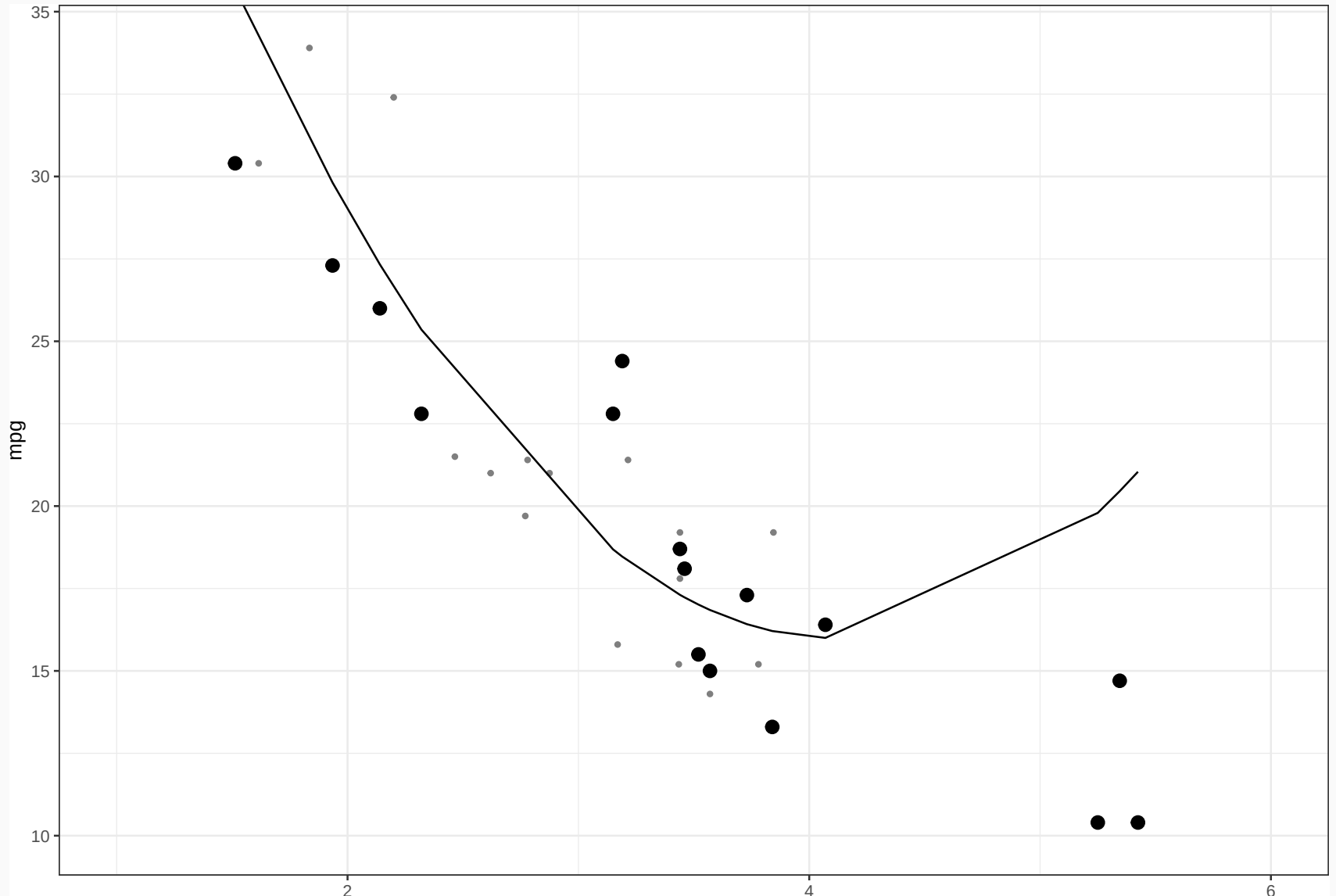
Checking the fit for our 6 degree polynomial model



Checking the fit for each of the models: compare to test data



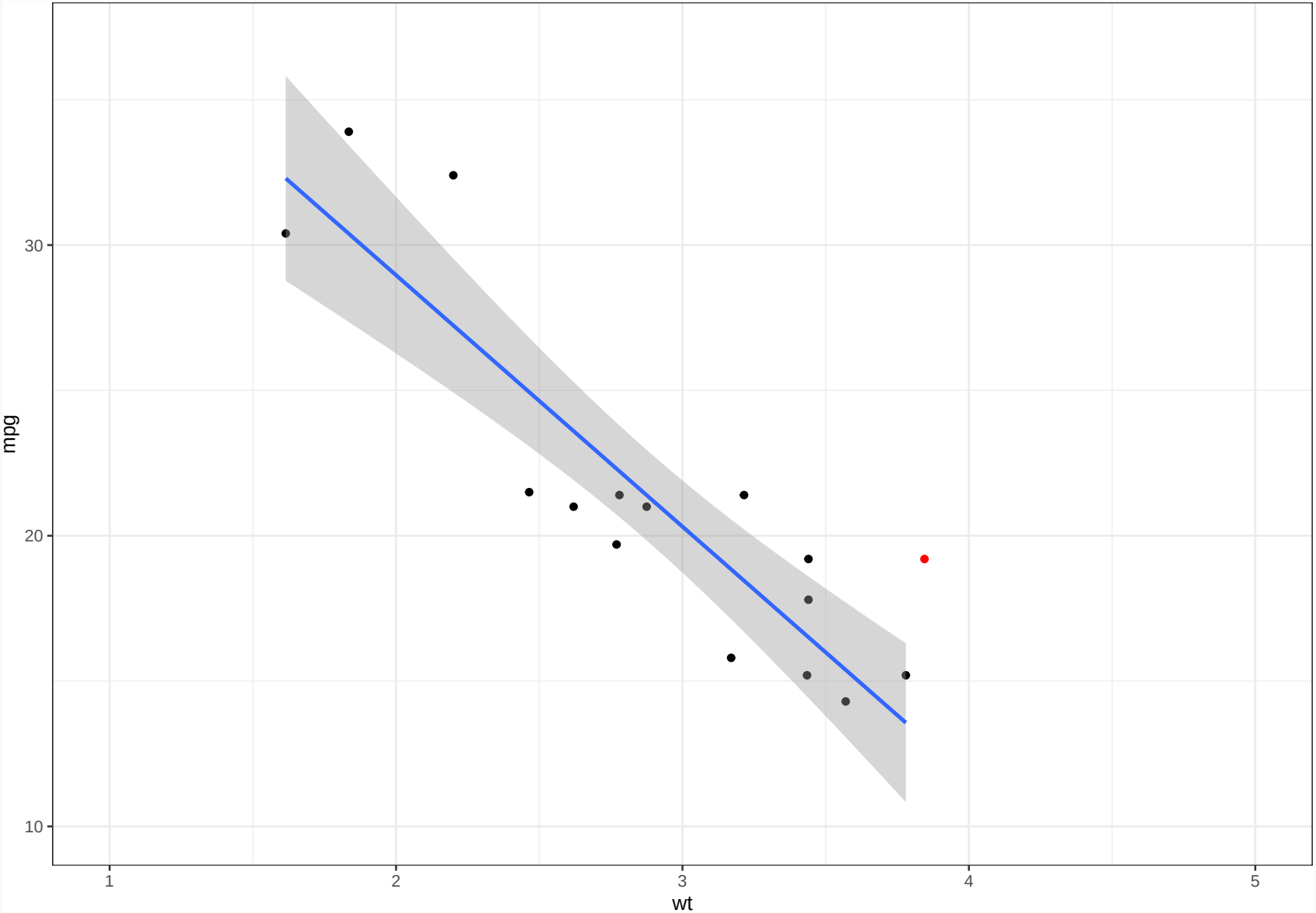
Checking the fit for each of the models: compare to test data, less complex model



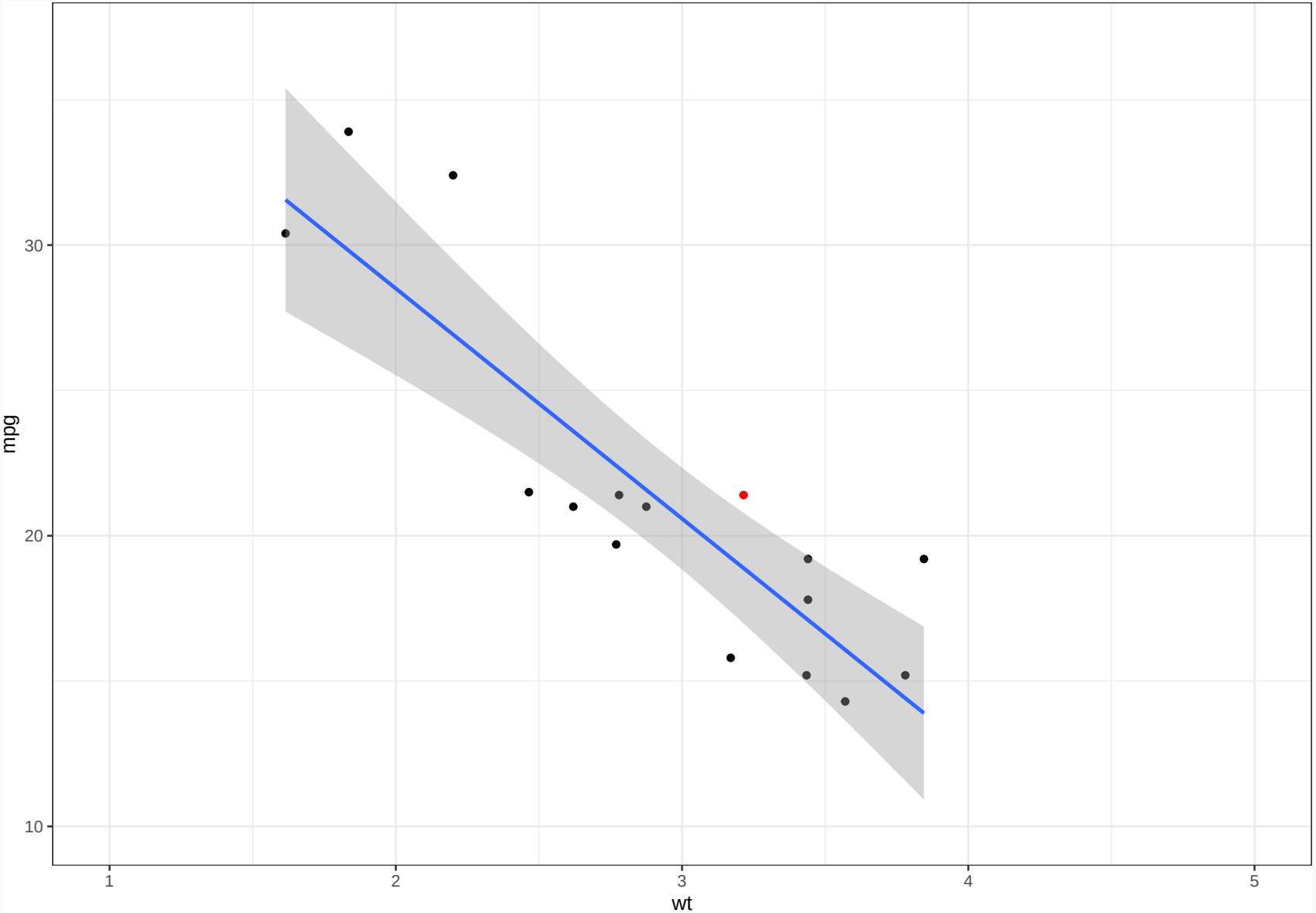
Leave-one-out cross validation

1. Define a model
2. Estimate the model, holding the first observation out
3. Predict the value for the held out observation, estimate prediction error
4. Repeat with 2:n observations
5. Average the error across each iteration

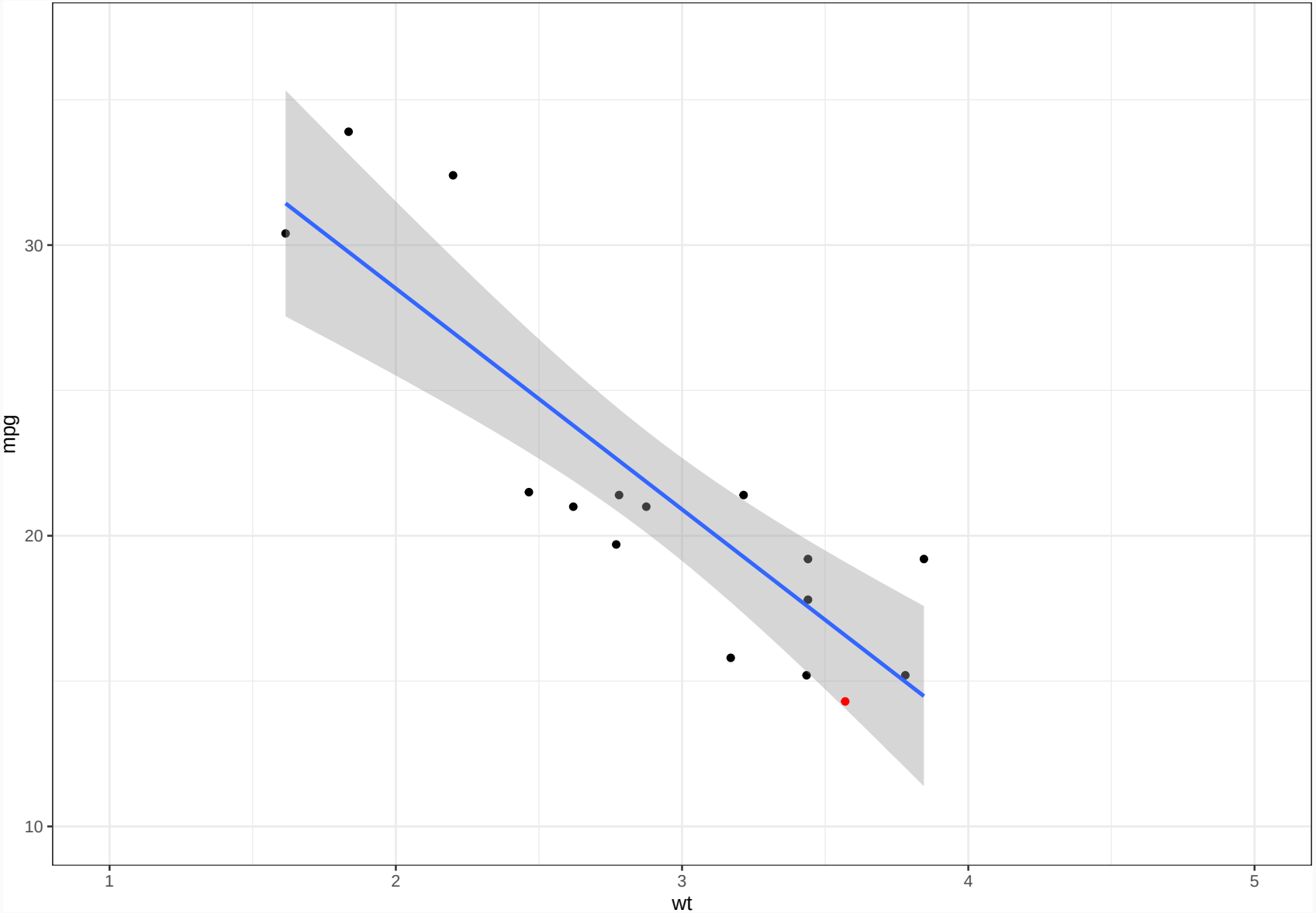
Leave-one-out cross validation



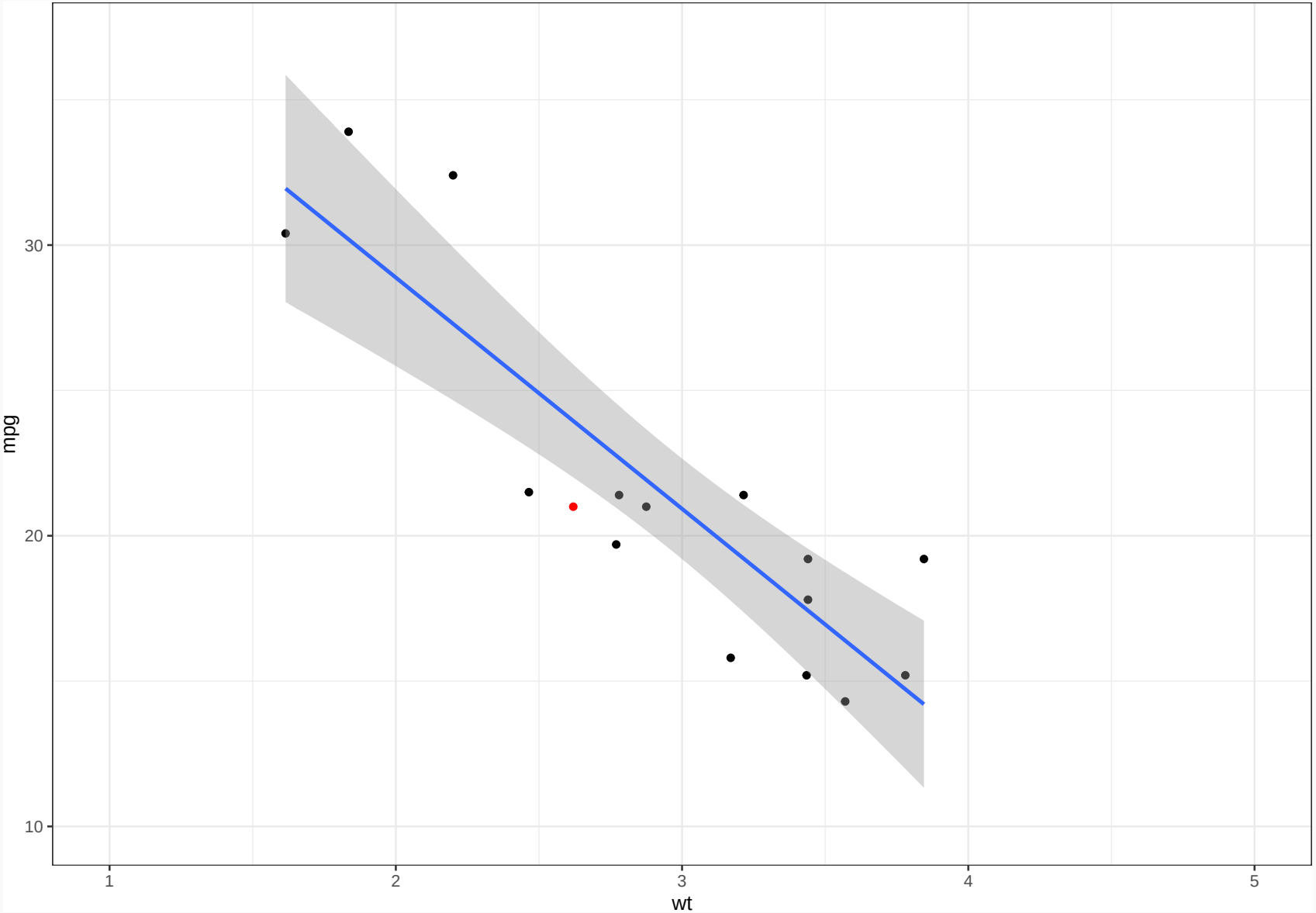
Leave-one-out cross validation



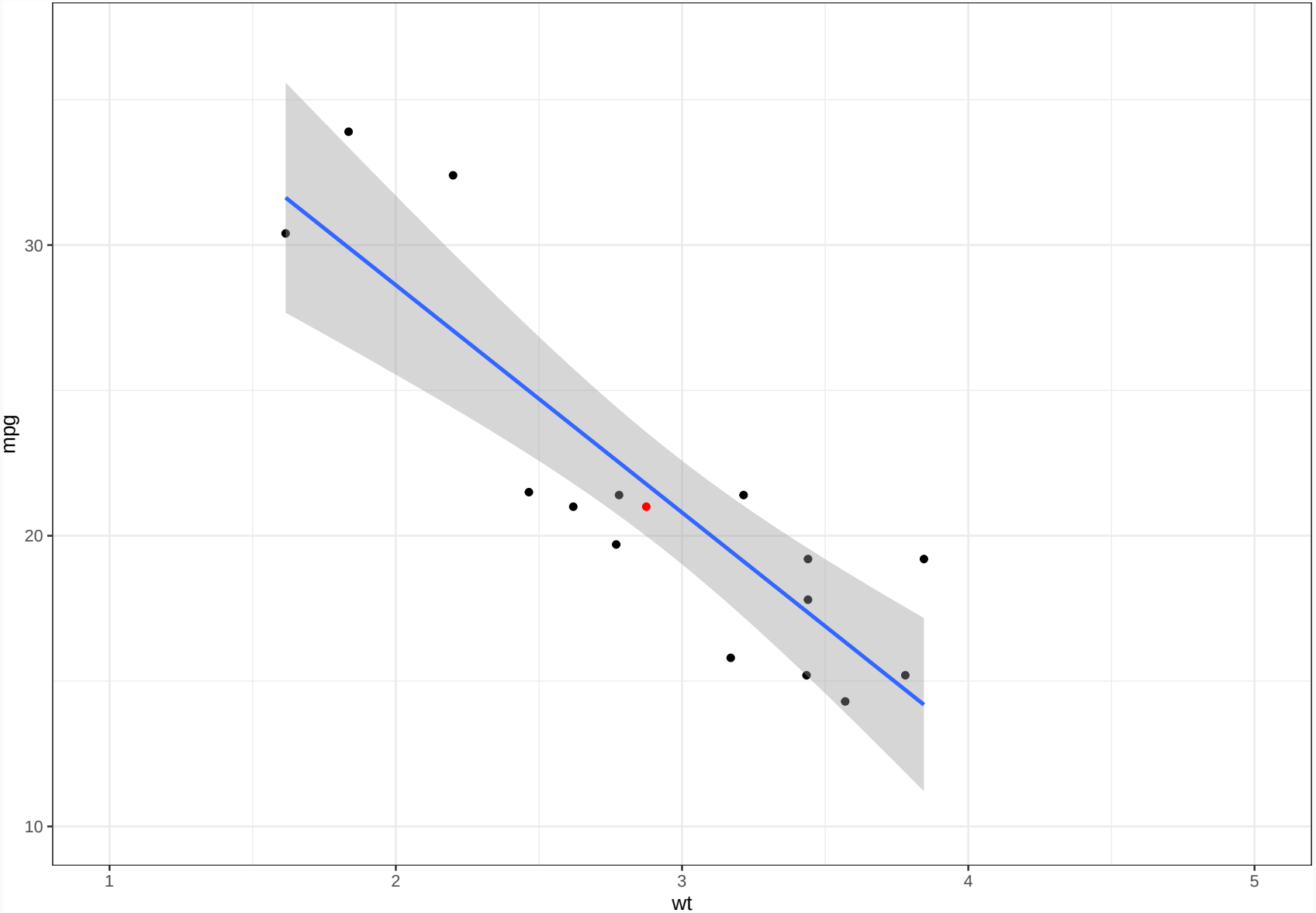
Leave-one-out cross validation



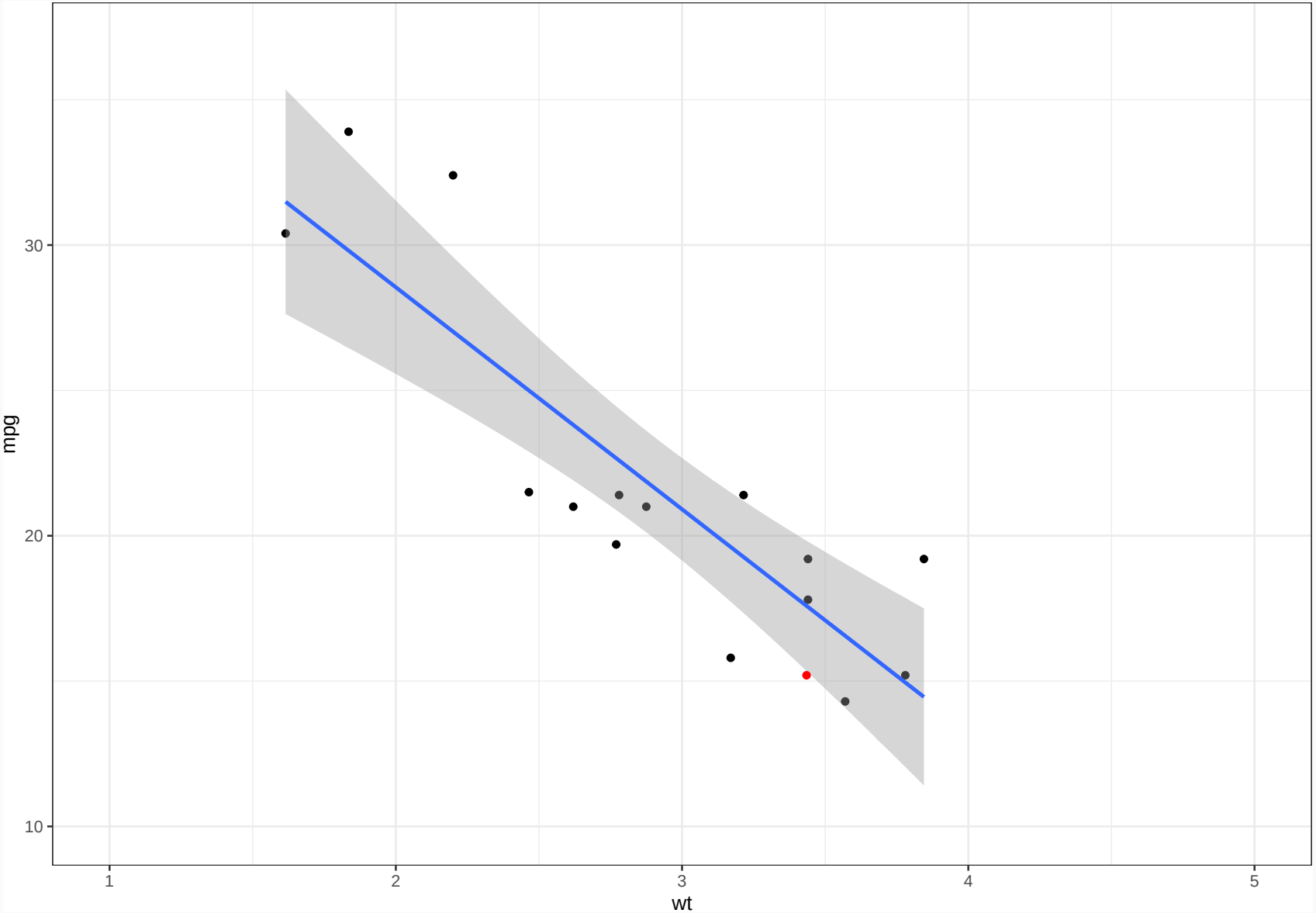
Leave-one-out cross validation



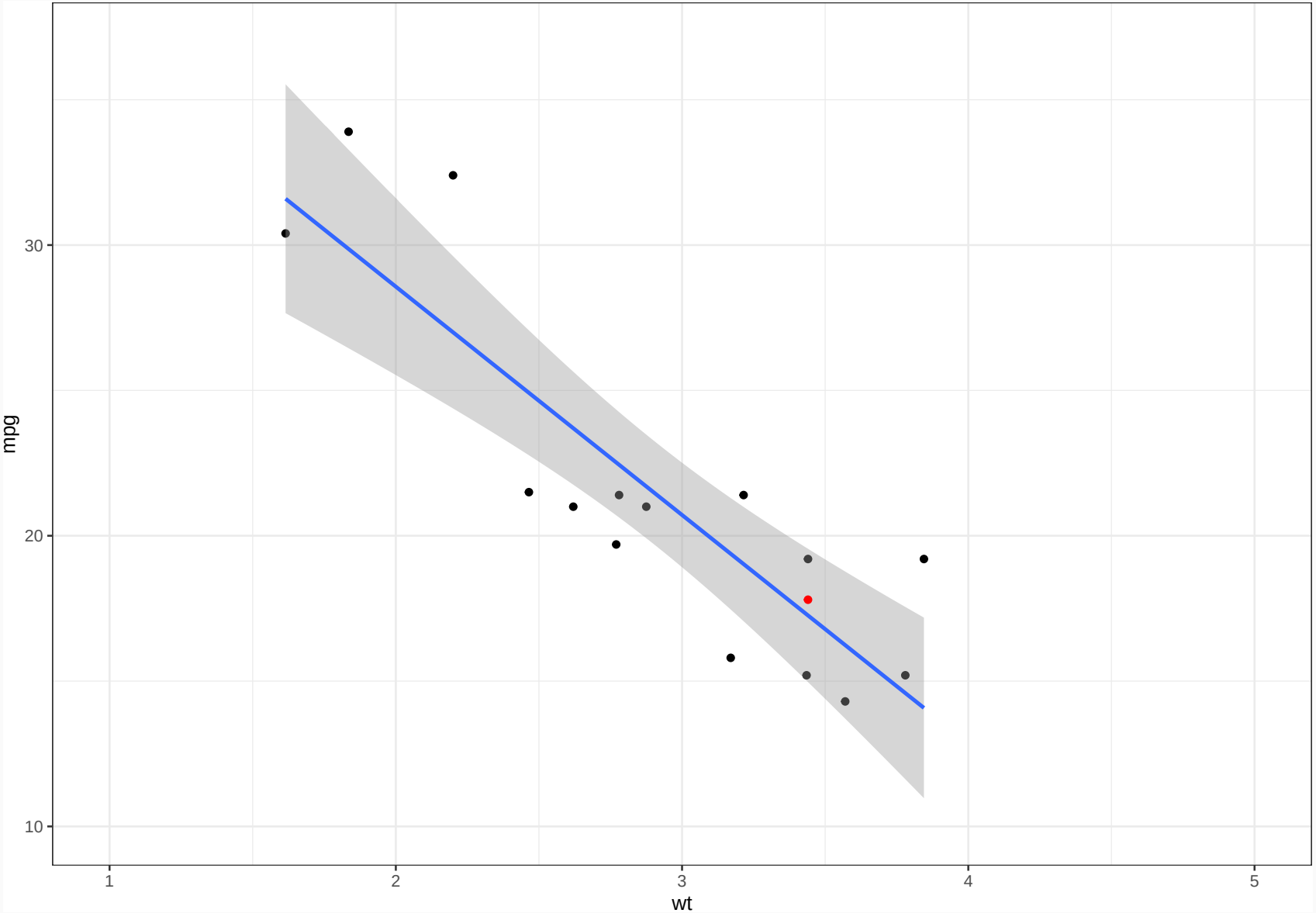
Leave-one-out cross validation



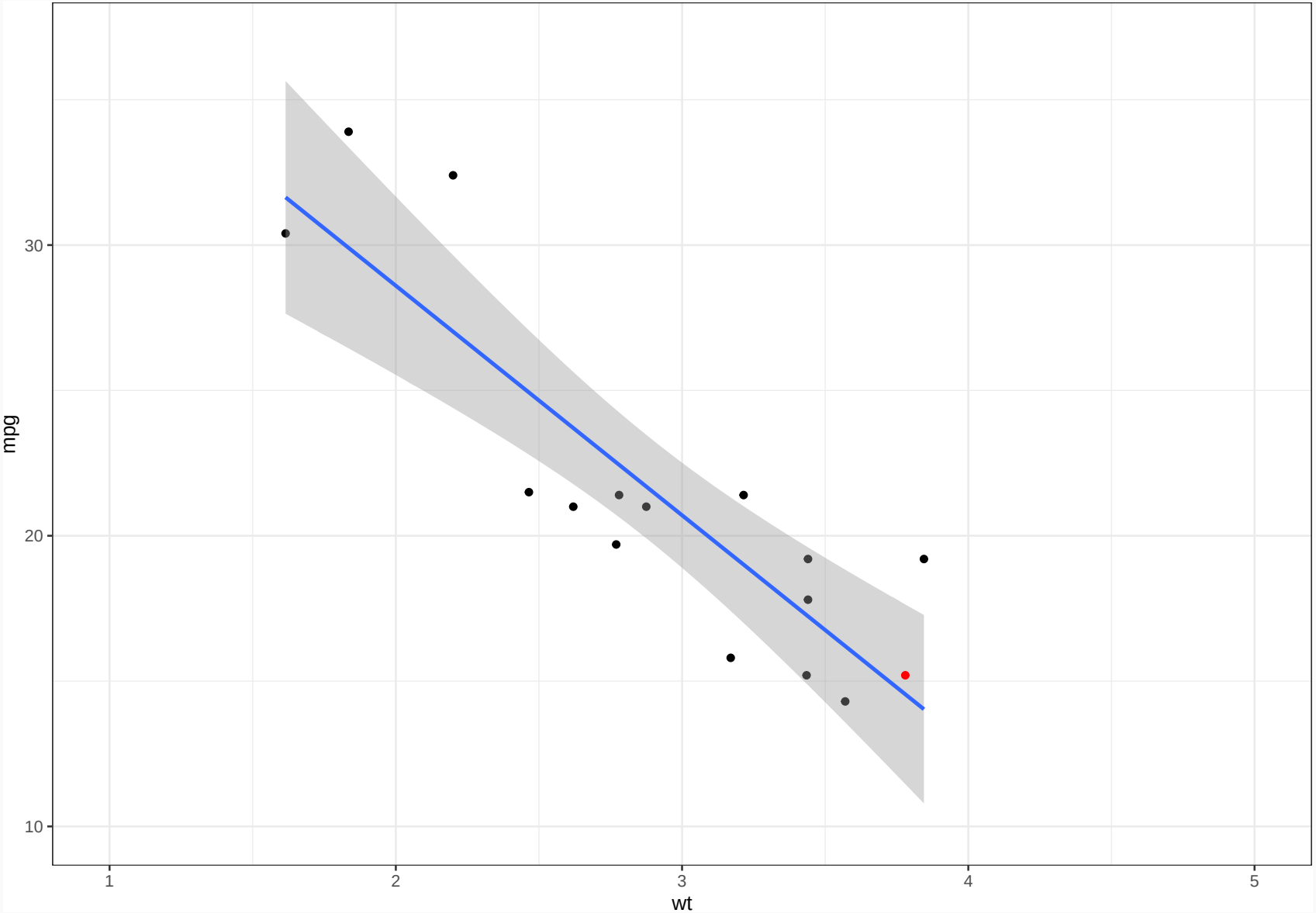
Leave-one-out cross validation



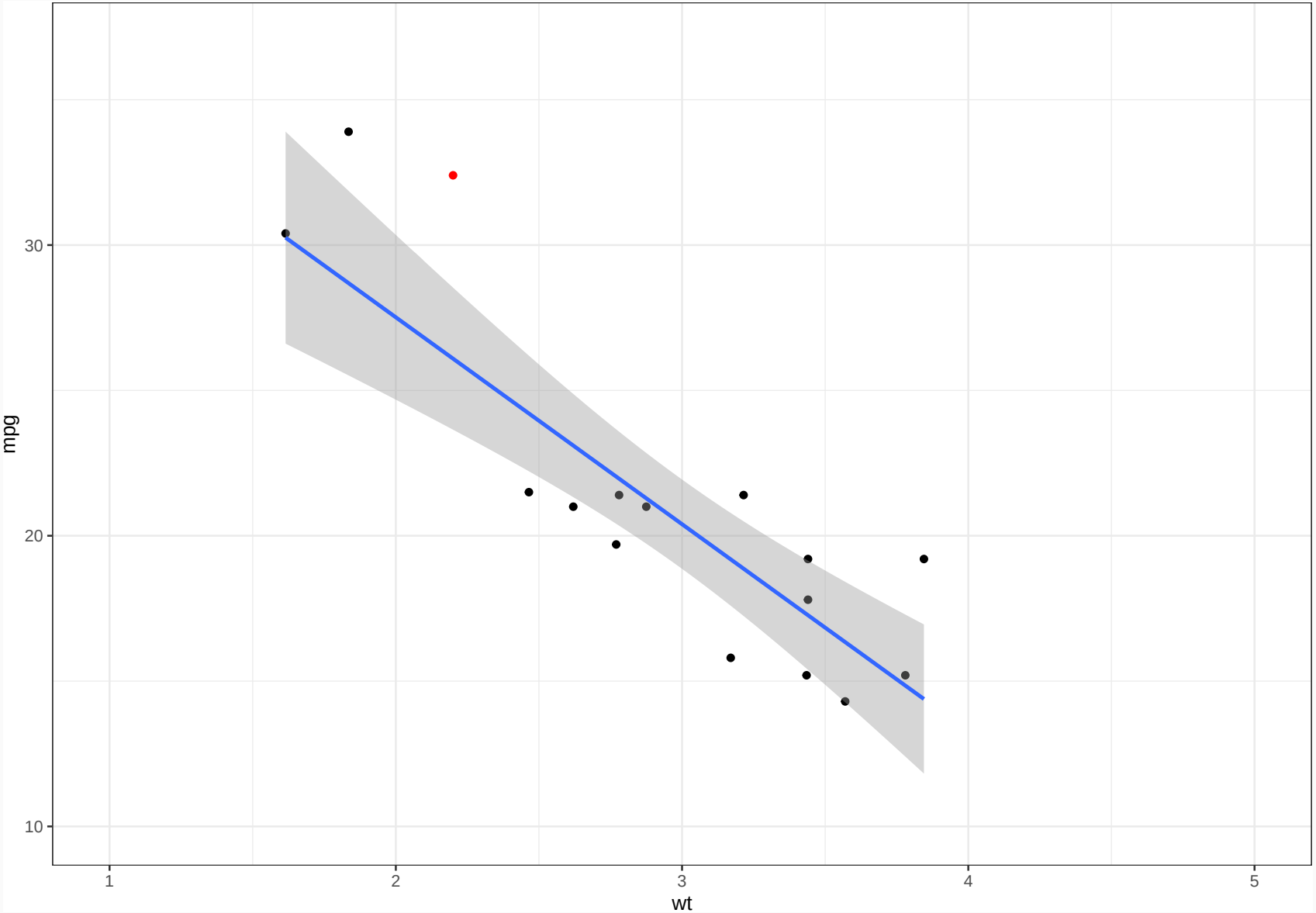
Leave-one-out cross validation



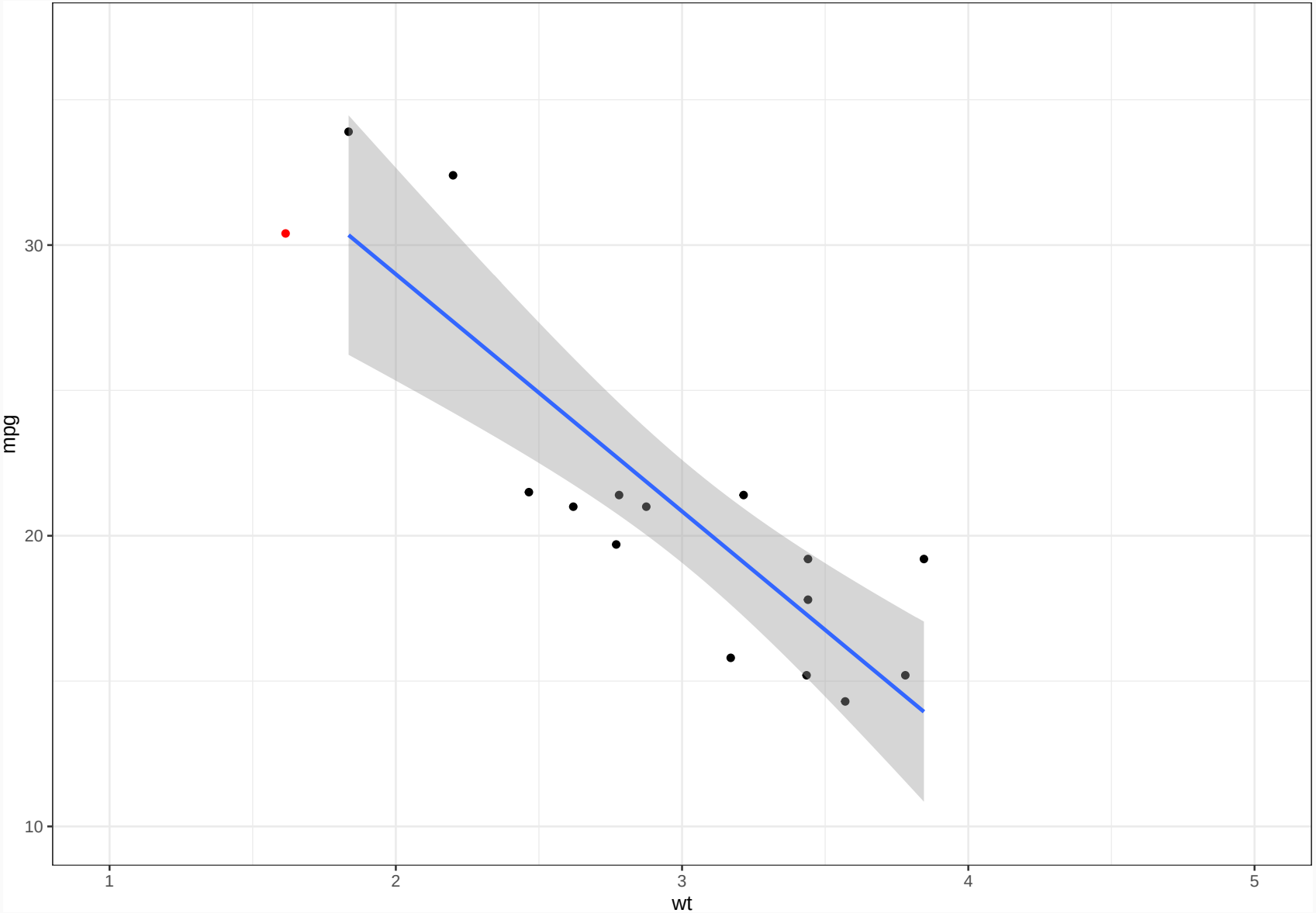
Leave-one-out cross validation



Leave-one-out cross validation



Leave-one-out cross validation



Implenting LOO-CV in R

```
cv_quap(mReg)
```

```
[1] -119.7317
```

PSIS provides an approximation of LOO-CV that is more computationally efficient

```
PSIS(mReg)
```

```
     PSIS       lppd  penalty  std_err
1 240.551 -120.2755 3.640157 14.12346
```

Model comparison using cross validation and information criteria

General approach

This approach is appropriate for comparing the estimated *predictive* accuracy of models. It is not appropriate for causal inference.

1. Estimate competing models
2. Compute the indicator of interest
3. Evaluate relative model predictive performance

Return to the plant fungus experiment

From the prior lecture and chapter 6: we conduct an experiment on the effect of an antifungal agent on plant growth. Conditioning on the presence of fungus confounds the relationship between the treatment and plant growth.

- fungus_0 is an intercept only model
- fungus_1 includes the treatment as a predictor
- fungus_2 includes both the treatment and fungus as a predictor

Compare these models using an information criterion

Information criteria compute an expected out-of-sample predictive score. AIC and BIC are routinely used, but better approaches now exist. We'll use WAIC (widely applicable information criteria).

In general, information criteria have two components:

$$IC = \log \text{ probability of model} - \text{penalty term}$$

The penalty term (crudely) penalizes models for additional complexity. For two models with the same fit (log probability), the IC will prefer the less complex model

Comparing model fits with WAIC

```
compare(fungus_0, fungus_1, fungus_2)
```

```
             WAIC       SE    dWAIC      dSE    pWAIC       weight
fungus_2 350.5533 16.30493  0.00000       NA 4.619920 1.000000e+00
fungus_0 395.8138 15.53685 45.26053 14.59953 3.180218 1.485251e-
10
fungus_1 395.9672 15.55988 45.41393 14.63406 3.272612 1.375598e-
10
```

- fungus_2 has the lowest WAIC score. This means that it is expected to predict new cases with the highest accuracy.
- The difference in WAIC between fungus_2 and fungus_1 is large and meaningful
- Note the dSE (difference standard error) value relative to dWAIC (the difference in WAIC between the lowest scoring model and all others)

Comparing model fits with WAIC

```
compare(fungus_0, fungus_1, fungus_2)
```

```
             WAIC       SE    dWAIC      dSE    pWAIC       weight
fungus_2 350.5533 16.30493  0.00000       NA 4.619920 1.000000e+00
fungus_0 395.8138 15.53685 45.26053 14.59953 3.180218 1.485251e-
10
fungus_1 395.9672 15.55988 45.41393 14.63406 3.272612 1.375598e-
10
```

- fungus_2 has the lowest WAIC score. This means that it is expected to predict new cases with the highest accuracy.
- The difference in WAIC between fungus_2 and fungus_1 is large and meaningful
- Note the dSE (difference standard error) value relative to dWAIC (the difference in WAIC between the lowest scoring model and all others)
- Information criteria don't care about causation!

Comparing model fits with PSIS

PSIS is an approximation of the LOO-CV method. It will warn you when there are influential observations that may bias the PSIS estimate. See 7.5.2 for a detailed example of how to address this using *robust regression*.

```
compare(fungus_0, fungus_1, fungus_2, func = PSIS)
```

```
             PSIS       SE    dPSIS       dSE    pPSIS       weight
fungus_2 350.9728 16.62359  0.00000        NA 4.844912 1.000000e+00
fungus_0 395.8844 15.58639 44.91165 14.82643 3.220821 1.768313e-
10
fungus_1 395.9048 15.50518 44.93200 14.78863 3.213985 1.750408e-
10
```

To start: maximum likelihood estimation

Conventional (frequentist) statistics usually uses MLE as its estimation engine. The basic logic is to solve for the value of a parameter under which the data is most likely.

For a binomial random variable

Let's simulate to see how it works.

$$y \sim N(0, 1)$$

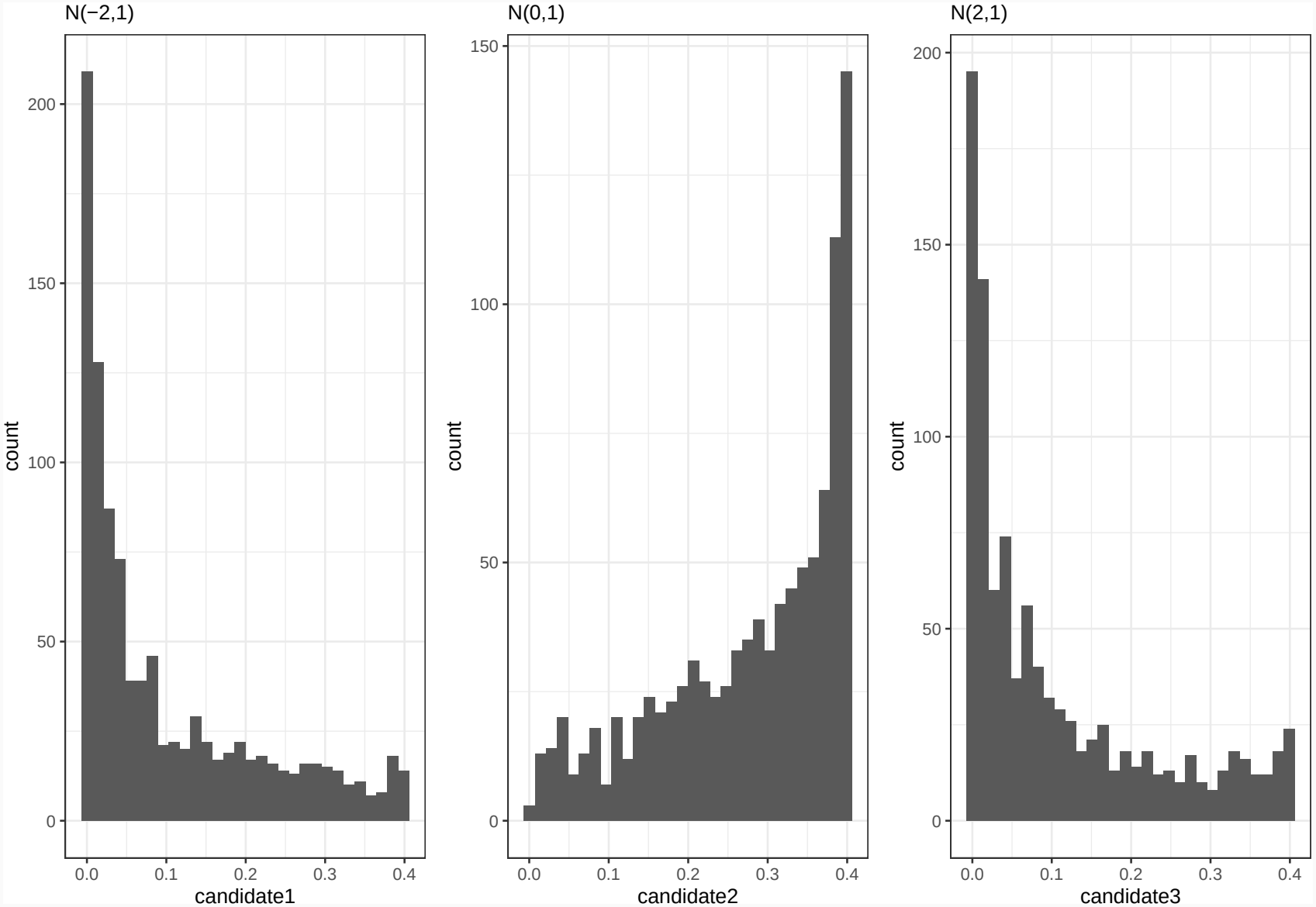
```
# take 1000 draws
y <- rnorm(1000, 0, 1)
```

What probability distributions could have given rise to the data?

Let's evaluate three candidate models: $y \sim N(-2, 1)$, $y \sim N(0, 1)$, and $y \sim N(2, 1)$

```
# compute the probability density for each observation in y
# under the candidate distribution
candidate1 <- dnorm(y, -2, 1)
candidate2 <- dnorm(y, 0, 1)
candidate3 <- dnorm(y, 2, 1)
pdat <- data.frame(candidate1, candidate2, candidate3)
```

The likelihood of each observation under each distribution



The log-likelihood

We can compute the logarithm of the joint likelihood of the data under each candidate model, then sum it up to get a relative plausibility

```
sum(log(candidate1))
```

```
[1] -3506.51
```

```
sum(log(candidate2))
```

```
[1] -1429.964
```

```
sum(log(candidate3))
```

```
[1] -3353.417
```

The most likely candidate has the largest log likelihood.

Limits of MLE

MLE works well for simple problems, but sometimes we don't know the exact shape of a candidate distribution, or want to estimate really complex models. For that, we can use MCMC

MCMC in action

We often don't know the exact shape of the posterior, so may not have a great idea for how to sample candidate distributions.

Recall $p(\theta|y) \propto p(y|\theta)p(\theta)$

Read as the posterior is proportional to the product of the likelihood and the prior.

Let’s poke around with a few MCMC algorithms

- Metropolis Hastings is the original
- Gibbs is a widely used improvement
- Stan uses Hamiltonian MC

Here’s a good demo: https://chi-feng.github.io/mcmc-demo/

Lab

- Simulate 10 x~N(20,10), then simulate 10 y~N(x, 20).
- Fit a linear model y~x with flat priors mu~dunif(-10000, 10000), sigma ~dunif(0, 1000), use only 100 iterations
- Inspect the traceplots

Lab

- Simulate 10 x~N(20,10), then simulate 10 y~N(x, 20).

```
x <- rnorm(5, 20, 10)
y <- rnorm(10, x, 20)
z <- x+y
dat <- data.frame(y, x, z)
```

- Fit a linear model y~x with flat priors beta~dunif(-10000, 10000), sigma ~dunif(0, 1000), use only 100 iterations

```
m0 <- brm(y~ x + z,
          prior = c(set_prior("uniform(-10000, 10000)", class = "b"
                    set_prior("uniform(0, 1000)", class = "sigma"))
          iter = 100,
          data = dat)
```

Lab

- Fit a linear model y~x with weakly informative priors beta~dnorm(0, 10), sigma ~dexp(1), use only 100 iterations

```
m0 <- brm(y~ x + z,
          prior = c(set_prior("normal(0,10)", class = "b"),
                    set_prior("exponential(1)", class = "sigma")),
          iter = 100,
          data = dat)
```

```
SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
Chain 1:
Chain 1: Gradient evaluation took 1.9e-05 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition wo
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:
Chain 1: WARNING: There aren't enough warmup iterations to fit the
```

Lab: waffledivorce

Let's break brms:

Fit a linear model divorce ~ Marriage * MedianAgeMarriage * South * WaffleHouses/Population

```
data("WaffleDivorce")
mw <- brm(Divorce ~ Marriage * MedianAgeMarriage * South * I(Waffle
```

```
SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
Chain 1:
Chain 1: Gradient evaluation took 1.2e-05 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition wo
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:
Chain 1: Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 1: Iteration:  200 / 2000 [ 10%]  (Warmup)
```